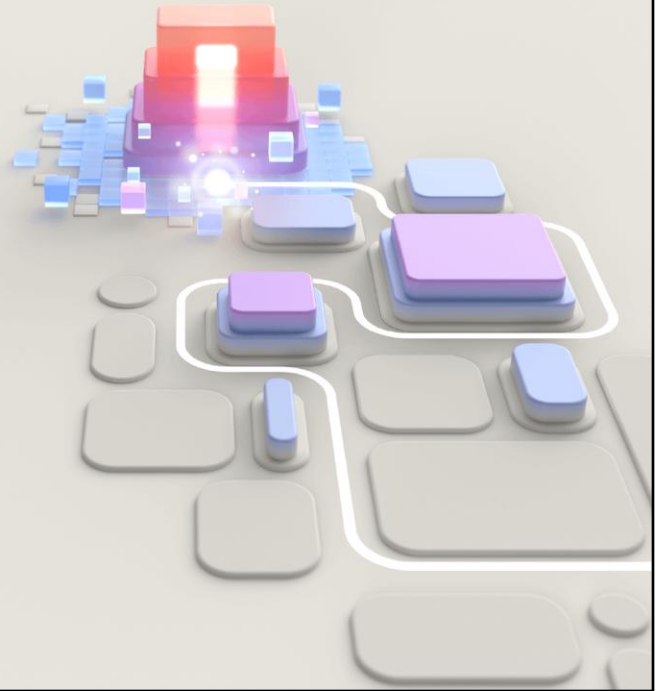




PL-400T00 Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.





Supplementary Learning Path 2: Create model-driven apps

© Copyright Microsoft Corporation. All rights reserved.

This learning path covers creating model-driven apps based on the data model in Microsoft Dataverse

The learning path covers the component-based approach to creating forms and views and then assembling the components into an app

This learning path is optional as it is **as these tasks are not the focus of the training/exam per the latest update to Objective Domain.**

Modules



- Get started with model-driven apps
- Configure forms
- Configure views
- Command bar
- Optimizing performance

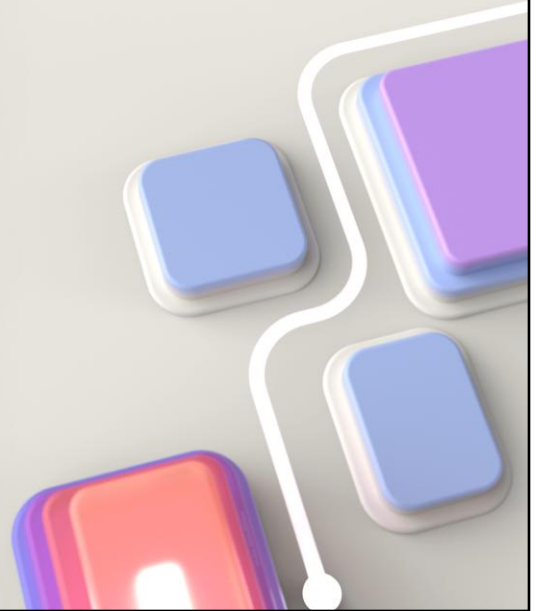
© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Get started with model-driven apps
2. Configure forms
3. Configure views
4. Command bar
5. Optimizing performance

Module 1: Get started with model-driven apps

© Copyright Microsoft Corporation. All rights reserved.



Design and create model-driven apps

Model-driven app design is an approach that focuses on quickly adding components to your apps. These components include dashboards, forms, views, and charts. With little or no code, you can make apps that are simple or complex.

In this module we will

- Introducing model-driven apps
- Building blocks of model-driven apps
- Design model-driven apps

Learning objectives

- Learn about model-driven app design
- Create a model-driven app

Learning Objectives

After completing this module, you will be able to:

- 1 Model-driven app design
- 2 Creating a model-driven app

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Get started with model-driven apps in Power Apps

<https://learn.microsoft.com/training/modules/get-started-with-model-driven-apps-in-powerapps>



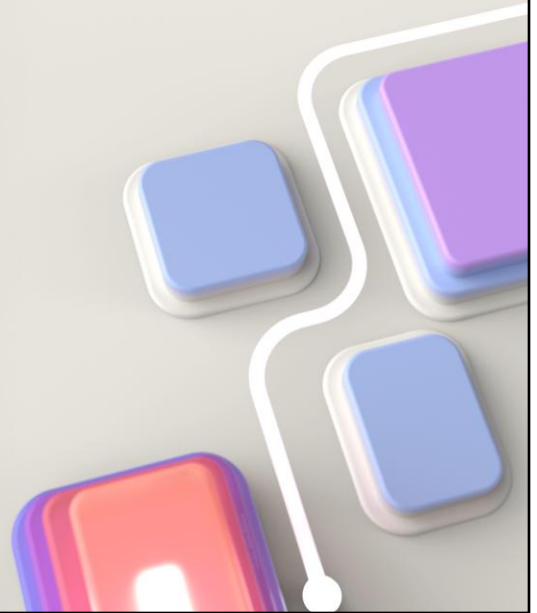
© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

<https://learn.microsoft.com/training/modules/get-started-with-model-driven-apps-in-powerapps>

What kind of apps have you made or used?

© Copyright Microsoft Corporation. All rights reserved.



Discussion

Engage students in a discussion of personal apps they have built or thought about building. Bring up the ones you have made to help lead the conversation.

Types of Power Apps

Model-driven

- Targets internal facing users
- Designed around the data
- Powered by Dataverse
- Automation
- Customizable user experience
- Customizable with or without professional developers

Canvas

- Targets internal facing users
- Designed around the user experience
- Connect to hundreds of data sources
- Customizable with or without professional developers

Portal (Power Pages)

- Targets external stakeholders
- Public-facing website
- Powered by Dataverse
- Customizable with or without professional developers

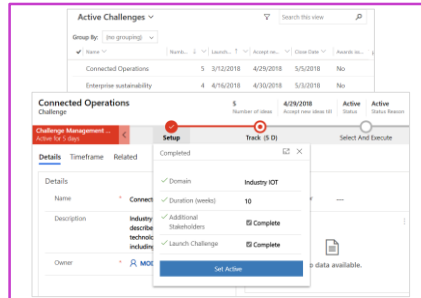
© Copyright Microsoft Corporation. All rights reserved.

In canvas apps, the app maker has total control over the app layout.

In model-driven apps, on the other hand, much of the layout is determined by the components you add. The emphasis is more on quickly viewing your business data and making decisions instead of on intricate app design.

Model-driven apps

- List and forms over data
- Consistent experience across apps and devices
- Easy to build with configuration first approach
- Model-driven apps require Microsoft Dataverse
- You can build more than one model-driven app from a single Dataverse environment



© Copyright Microsoft Corporation. All rights reserved.

Unlike canvas apps, where you build out an app screen-by-screen by adding logic and code as you go, model-driven apps can be created with a few simple steps. Model-driven apps use a component-focused approach to develop the app. When developing canvas apps, you have complete control over the appearance and behavior of your app, whereas with model-driven apps, the layout is mainly based on the components that you add to the app.

With model-driven apps, a number of different components and component properties are available for you to add and modify when designing an app.

Model-driven app design provides the following benefits:

- Rich component-focused, no-code design environments
- An ability to create complex responsive apps with a similar UI across a variety of devices from desktop to mobile
- Robust design capability
- Apps can be distributed as a solution

Built-in to all model-driven apps

Advanced Find	Searching	Personal Views	Personal Dashboards
Editable Grids	Excel Integration	Client Dev Extensibility	Client-side Business rules
Reusable Quick View Forms	Business Process Control	Timeline Activity View	Command Bar Extensibility
Duplicate Detection	Multiple Makers	Recently Viewed/Pinned	Localization support

© Copyright Microsoft Corporation. All rights reserved.

Multiple Makers – Canvas apps can only have one person editing at a time,
Model-driven apps can have multiple makers working on different parts of the app
Strong ISV ecosystem of model-driven apps
Full support of solutions

Demo: Model-driven apps

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Take a tour of a model-driven app, use one of your own or use one of the sample apps included with Power Apps.

Show navigation, forms, views, etc.

App building blocks

- 1 Data:** Define the data the app will be based upon
- 2 User interface:** Determine how users interact with the app
- 3 Logic:** Determine what business processes, rules, and automation the app will have
- 4 Visualization:** Determine what type of data and reporting the app will show

© Copyright Microsoft Corporation. All rights reserved.

When creating a model-driven app, you can use tables from Microsoft Dataverse as your building blocks. Model-driven apps start with your data model, building up from the shape of your core business data and processes in Dataverse to model forms, views, and other components.

It's important to ensure that your business data and business processes at the data level are structured properly before you compose your app. Model-driven apps will automatically generate a UI that is responsive across devices; however, this outcome relies heavily on how your data is modeled in Dataverse.

User interface

Determine how users will interact with the app:

- 1 App:** Apps determine the app fundamentals, like components, properties, and the URL
- 2 Site Map:** Specifies the navigation of your app
- 3 Form:** Include a set of data entry columns for a given table
- 4 View:** Defines how a list of rows for a specific table appear in your app

© Copyright Microsoft Corporation. All rights reserved.

Apps determine the app fundamentals, like components, properties, the client type, and the URL.

A site map specifies the navigation for your app.

Forms include a set of data entry columns for a given table. This set of data entry columns matches the items that your organization tracks for the table. One example is a set of data entry columns where users enter relevant information to track a customer's previous orders together with specific requested reorder dates.

Views define how a list of rows for a specific table appears in your app. A view defines the columns shown, the width of each column, the sort behavior, and the default filters.

Visualization

Determine what type of data and reporting the app will show:

- 1** **Chart:** Individual graphical visualizations that can appear in a view or a form or can be added to a dashboard
- 2** **Dashboard:** Show one or more graphical visualizations that provide an overview of actionable business data
- 3** **Embedded Microsoft Power BI:** Adds embedded Power BI tiles and dashboards to your app

© Copyright Microsoft Corporation. All rights reserved.

Charts are individual graphical visualizations that can appear in a view or a form or that can be added to a dashboard.

Dashboards show one or more graphical visualizations that provide an overview of actionable business data.

Power BI adds embedded Power BI tiles and dashboards to your app. Power BI is a cloud-based service that provides business intelligence (BI) insight.

Control security when sharing model-driven apps

Model-driven apps use role-based security for sharing

- 1 Access to the app
- 2 Access to tables in the app
- 3 Access to model-driven app features

© Copyright Microsoft Corporation. All rights reserved.

Model-driven apps use role-based security for sharing. The fundamental concept in role-based security is that a security role contains privileges that define a set of actions that can be performed on tables within the app. This approach means that while two people are able to use the app, one user might only be able to read records, or records that they themselves created. The other user might be able to see all records and have the rights to delete those records.

All app users must be assigned one or more predefined or custom security roles. Or, security roles can be assigned to teams. When a user or team is assigned to one of these roles, the person or team members are granted the set of privileges associated with that role.

The process for sharing a model-driven app is different from sharing a canvas app. Model-driven app sharing depends on how the Microsoft Dataverse data table privileges are assigned for the tables that are in the app.

<https://docs.microsoft.com/en-us/power-apps/maker/model-driven-apps/share-model-driven-app>

Module 2: Configure forms

© Copyright Microsoft Corporation. All rights reserved.

Configure forms

In this module, you'll learn about forms, grids, views, charts, and dashboards that can be used in model-driven apps.

In this module we will

- Forms overview
- Form elements
- Configure multiple forms
- Use specialized form components

Learning objectives

- Learn about form elements and controls
- Configure forms
- Use specialized form components

Learning Objectives

After completing this module, you will be able to:

- 1 Use and configure form elements and controls
- 2 Identify views and use public views
- 3 Learn how to configure charts and dashboards

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Configure forms, charts, and dashboards in model-driven apps

<https://learn.microsoft.com/training/modules/configure-model-driven-apps-customer-engagement-apps>



© Copyright Microsoft Corporation. All rights reserved.

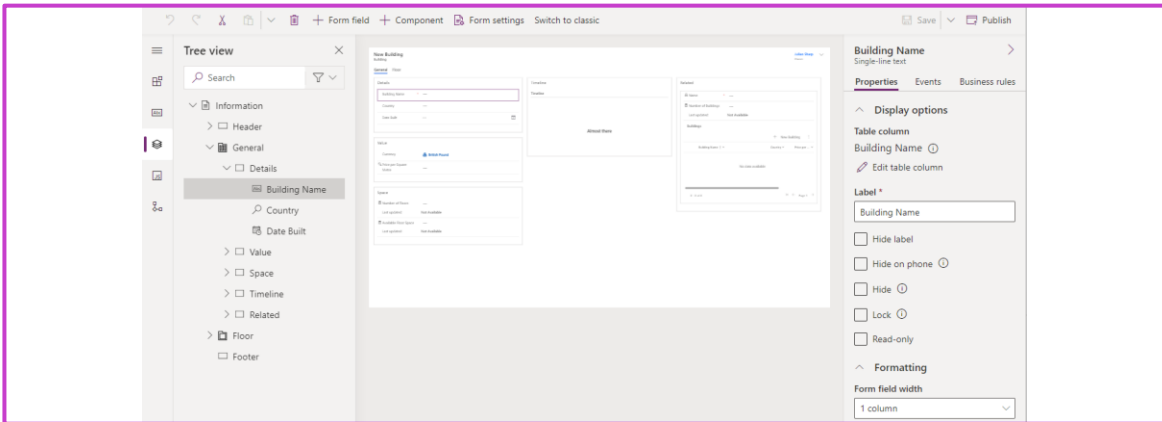
Link to student courseware on Learn

<https://learn.microsoft.com/training/modules/configure-model-driven-apps-customer-engagement-apps>

This Learn module is used for this less and the next one on views

Form designer

Model-driven forms are how data for a row is presented to the user during editing and viewing.



© Copyright Microsoft Corporation. All rights reserved.

Model-driven forms are how detailed data for a row is presented to the user during editing and viewing. Forms provide a structured way to represent the data when it's rendered for interaction with the user. Forms abstract the form creator from the exact rendering specifics required to transform the form definition you create for the user's device size and capabilities. However, when you lay out the form using some knowledge of the current form rendering engine it will help you make more usable forms. Unified Interface is the current name of the framework that renders the form definitions for the user. When editing forms, the changes won't be visible to users until the form is saved and published.

Forms are organized into header, body, and footer, each capable of containing form elements like columns. The body of the form is further structured with areas called tabs that contain sections. Tabs and sections can be configured to support columns of form elements giving further structure to the content. The first tab on a form is the most important and should contain the priority data the user should see. While you could configure many tabs, keeping it to a smaller number with logical grouping of data can make a more usable experience because the user isn't constantly tabbing around to find things.

The most common task you'll perform when editing a form is placing column controls on the form. A column can be added to the form multiple times if needed, the value shown will be the same for each occurrence. Each control you place on the form you can control the label shown, and other properties like visibility and read only. Without special configuration, a column will render with a control automatically selected by the runtime that is appropriate for the data type of the column. For example, an Option Set column will show the data in a drop-down list.

The following shows an example of the column properties that can be edited on each column control.

Depending on the column type, the column might offer other settings that can be changed. For example, a Lookup column will allow you to set the default view and other view-related options.

Form structure

Model-driven app forms have a defined structure

- 1 Tabs
- 2 Sections
- 3 Controls

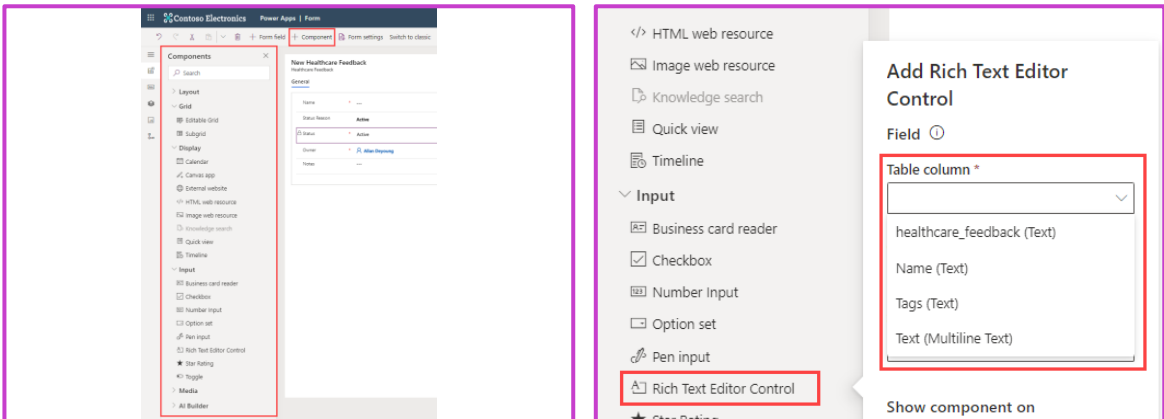
© Copyright Microsoft Corporation. All rights reserved.

Model-driven forms provide the means for presenting a row of detailed data to the user for editing and viewing. Forms give you a structured way to represent the data rendered for user interaction. Forms abstract the form creator from the exact rendering specifics required to transform the form definition you create for the user's device size and capabilities. However, when you lay out the form using some knowledge of the current form rendering engine it will help you make more usable forms. Unified Interface is the name of the framework that renders the form definitions for the user. When you edit forms, the changes won't be visible to users until the form is published.

Forms are organized into **header**, **body**, and **footer**, each capable of containing form elements, like **columns**. The body of the form is further structured with areas called **tabs** that contain **sections**. Tabs and sections can be configured to support columns of form elements giving further structure to the content. The first tab on a form is the most important and should contain the priority data the user should see. While you could configure many tabs, keeping it to a smaller number with logical grouping of data can make a more usable experience because the user isn't constantly tabbing around to find things.

Form component controls

Components can be added to columns on the forms to display how the control is displayed

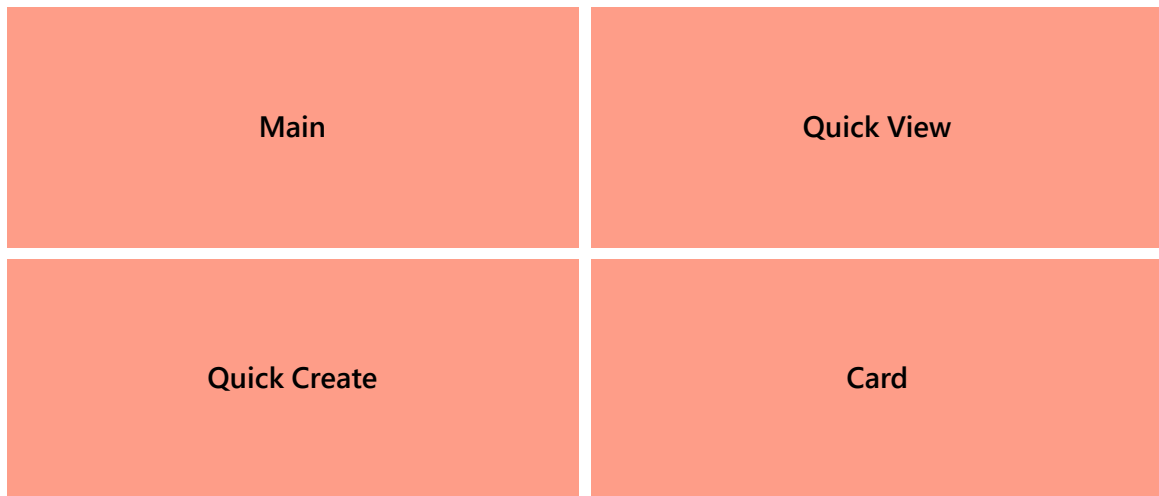


© Copyright Microsoft Corporation. All rights reserved.

Form Components allow you the ability to further customize your form. The Layout choices provide you ways to organize the physical structure of your form. Grid components enable you to allow data entry via a grid. Display components allow you to insert calendars, external websites, Canvas apps and other items into the form experience. Input components provide your user with other means for data entry, such as checkboxes, option sets, toggles and pen inputs.

When you select a component to add, Power Apps will prompt you for which field you could apply that component to, and to annotate which platforms to display the component on. If there's no table column that you can apply the component to, you won't be able to insert it. In the example below, we've selected a **Rich Text Editor Control**. We're prompted to select a **Table column** (notice that it's a required field) and the dropdown provides us four options to choose from.

Form types



© Copyright Microsoft Corporation. All rights reserved.

Main

Main forms are the primary user interface where people view and interact with their data. Main forms provide the widest range of options and are available for model-driven apps. One of the primary design objectives for main forms is that you design them once and deploy them everywhere. The same main form you design for a model-driven app or the Dynamics 365 customer engagement web application, is also used in Dynamics 365 for Outlook and Dynamics 365 for tablets.

The advantage to this approach is that you don't have to integrate changes into multiple forms.

Quick View

A quick view form can be added to another form as a quick view control. It provides a template to view information about a related table row within a form for another table row. This means your app users don't need to navigate to a different row to see the information needed to do their work. Quick view controls are associated with a lookup column that is included in a form. If the lookup column value isn't set, the quick view control won't be visible. Data in quick view controls can't be edited and quick view forms don't support form scripts.

Quick Create

With quick create forms, your app can have a streamlined data entry experience with full support for logic defined by form scripts and business rules.

In Dynamics 365, quick create forms appear when you select the **Create** button in the navigation bar or when you choose **+ New** when creating a new record from a lookup or sub-grid.

The Dynamics 365 customer engagement mobile apps use quick create forms for creating new records. If an entity already has a quick create form configured for it, the mobile apps use that form. If an entity doesn't have a configured quick create form, Dynamics 365 generates a quick create form for creating records in the mobile apps based on the main form definition.

Card

These forms are used in the interactive dashboards to show the entity data in the streams of interactive dashboards.

Card - Card forms are used in views for model-driven apps. Card forms are designed to present information in a compact format that is suitable for mobile devices. Although card forms can be created and edited in the same way as other form types, card forms are added to apps differently. Instead of adding a form as an app component, custom card forms are added to views by using the Read Only Grid control.

Specialized form components

- Timeline
- Sub-grid
- Quick view
- Reference panel
- Timer control
- IFrame

The screenshot displays the configuration settings for a Timeline component in a Dynamics 365 form. The interface is divided into several sections:

- Timeline:** Includes a 'Name' field with a dropdown arrow, a 'Record types to show' section with checkboxes for 'Activities' and 'Notes', a 'Profile picture' dropdown, and an 'Advanced' section with 'Quick entry record type' (set to 'Notes'), 'Sort default' (set to 'Descending'), 'Display layout' (set to 'Roomy'), and 'Records shown on page' (set to '10').
- Filter pane:** Includes a checkbox for 'Enable filter pane' (checked), a checkbox for 'Expand filter pane by default' (unchecked), an 'Edit filter pane' link, an 'Additional settings' section with checkboxes for 'Hide Timeline label' (unchecked), 'Hide the date portion for any datetime in the past 24 hours' (checked), 'Enable search bar' (checked), 'Expand all records by default' (unchecked), and 'Enable "What you've missed" summary' (unchecked), and a 'Move timeline to a new tab for mobile apps' section with a 'Default' dropdown.
- Activities:** Includes a list of 'Activity types' with checkboxes: 'Appointment', 'Email', 'Fax', 'Letter', 'Phone Call', 'Recurring Appointment', 'Social Activity', 'Task', and 'Teams chat'. All are checked.
- Sort activities by:** A dropdown menu set to 'Last Updated'.
- Create activities using:** A dropdown menu set to 'Quick create form'.
- Open activities using:** A dropdown menu set to 'Main form'.
- Activity rollout type:** A dropdown menu set to 'Extended'.

© Copyright Microsoft Corporation. All rights reserved.

As mentioned previously, there are several specialized form components available for model driven apps. This topic contains additional details on several of the most common ones.

A reference panel is a specialized section that can be added to main forms that offers interaction with related data in the context of the hosting table row. It can contain sub-grids of related tables and display related data without having to navigate to a different form. To get the full value of a reference panel you must also have Quick View forms configured. Each form can only have a single reference panel, but each reference panel can contain multiple sub-grids.

Bing Maps can be displayed on a form for the account, contact, lead, quote, order, invoice, competitor, and system user tables.

Use a timer control with forms where rows need to meet a specific time-based milestone. A timer control shows how much time is available to complete an action in the resolution of an active row or how much time has passed since the time to complete the action has passed. At a minimum, timer controls must be configured to show success or failure in completing the action. In addition, they can be configured to display warnings when the conditions are approaching failure.

A quick view control on a model-driven app form displays data from a row that is selected in a lookup on the form. The data displayed in the control is defined using a quick view form. The data displayed is not editable, but when the primary column is included in the quick view form, it becomes a link to open the related row.

A timer control can be added to a form for any table, but they are most frequently used for the case table, especially when linked to columns that track service level agreements. You can add multiple timer controls in the body of a form. You can't add them to the header or footer.

You can add iFrames to a form to integrate content from another website within a form.

The Timeline control provides a combined place to view customer interactions including activities and notes all in one location. The timeline quickly allows you to see when the interaction occurred and what is new since last visit to the row. Using filters users can quickly trim down what is visible. The control properties allow you to change things like what the default create row type is, as well as what activities show in the timeline.

Demo: Form designer

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Tour of form designer

Main form

Add sub-grid

Quick View form

Configure multiple forms

Users of model-driven apps will encounter many forms in their user experience, and will even encounter different forms for the same table depending on their role

You can create more than one Quick Create form for a given table, but only the first one in order will be available for end users.

Security roles

Form order

Fallback forms

Form access checker

Form order for Account table

When multiple forms are enabled for a user, choose the order in which they will appear. Ideally, we recommend only having one form per security role. [Learn more](#)
Drag and drop to change the order or use the keyboard (Tab/Tab+Shift to navigate the list, Space to select/release a form, Up/Down to move the form).

Choose a form set

Main Form

Name

:: Account

:: Account for Interactive experience

:: Information

Save and publish

Cancel

© Copyright Microsoft Corporation. All rights reserved.

Form settings allow us to install controls on our forms that work behind the scenes, without our user's notice. We can define security roles to the form to control access to it, the form order, and fallback forms. Additionally, Form settings provide us the ability to check a particular security group's access to the form. The image below shows the *Security roles* tab.

Users of model driven applications will encounter many forms in their user experience, and will even encounter different forms for the same table depending on their role, job tasks and desired outcomes.

You can set form order when using multiple forms. This will define the order in which a user sees the available forms, within the allowed forms for their security roles.

Enable for Fallback

[!Note] You can create more than one Quick Create form for a given table, but only the first one in order will be available for end users.

Access to forms

- 1** Each table must have a designated fallback form. This is the form that all users would see if they do not have a security role assigned that matches your role tailored forms.
- 2** Only Main forms can be assigned to security roles.
- 3** When a user has access to multiple forms, a form selector will be available near the top of the form.
- 4** You can designate a main form as inactive. This will make it not visible to all users, regardless of security roles.
- 5** The table definition must allow for Quick Create before you will be able to configure a Quick Create form.

© Copyright Microsoft Corporation. All rights reserved.

Each table must have a designated fallback form. This is the form that all users would see if they do not have a security role assigned that matches your role tailored forms.

Only Main forms can be assigned to security roles.

When a user has access to multiple forms, a form selector will be available near the top of the form. If a user has access to only a single form for a given table, there will be no form selector visible.

You can designate a main form as inactive. This will make it not visible to all users, regardless of security roles. This feature was included primarily to manage new forms included when organizations upgrade but you can use it to prevent people from being able to use any main form.

[!Note] Controlling user access to forms is not necessarily a secure means to prevent access to data. Sometimes users have other ways to interact with data such as advanced find or background automation.

Module 3: Configure forms

© Copyright Microsoft Corporation. All rights reserved.

Configure views

In this module, you'll learn about configuring views that can be used in model-driven apps.

Learning Objectives

After completing this module, you will be able to:

- 1 Use and configure views
- 2 Identify views and use public views

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Configure forms, charts, and dashboards in model-driven apps

<https://learn.microsoft.com/training/modules/configure-model-driven-apps-customer-engagement-apps>



© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

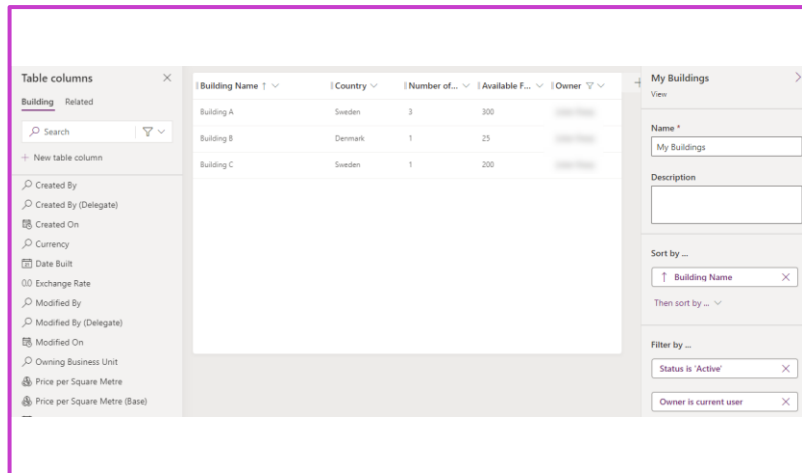
<https://learn.microsoft.com/training/modules/configure-model-driven-apps-customer-engagement-apps>

This Learn module is used for this less and the next one on views

Overview of views

Views are lists of data from a Dataverse table

- Columns from table
- Columns from many-to-one relationships
- Sorting
- Filtering



© Copyright Microsoft Corporation. All rights reserved.

Tables come with 2 public views by default

Public vs Personal views

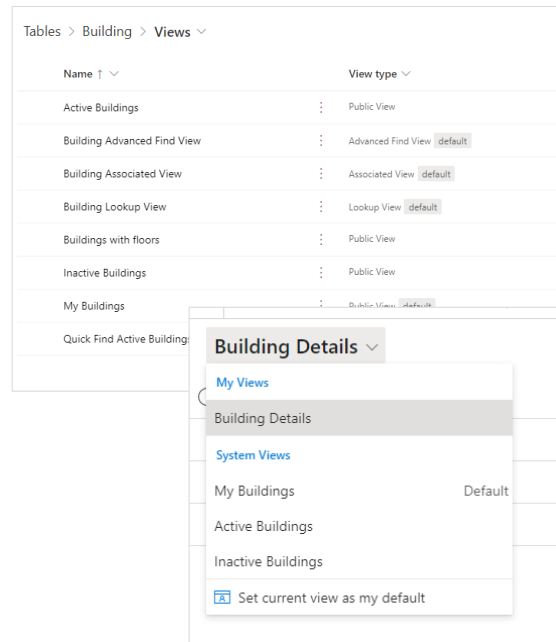
Public (System) views are managed in the Maker portal

- Public views are not controlled by security
- Public views can be turned off
- Public views can be included/excluded in an app

Personal (My) views are created in a model-driven app

- Personal views can be shared with users and teams

© Copyright Microsoft Corporation. All rights reserved.



Note: Difference with forms that can be controlled by security roles

Specialty views

There are four types of specialty view;

Lookup - The first three columns of the lookup view are visible when a user clicks on the magnifying glass icon on the related lookup column

Associated -Users view data on rows from the host table but can also view related tables

Quick Find - Displays search results and defines which columns are used to determine search results

Advanced Find - When a user is making a personal view or doing an improvised advanced search, this will be the default view of the search results

© Copyright Microsoft Corporation. All rights reserved.

In addition to the standard views already covered in this module, there are several specialty views available known as public views. These public views are automatically created for custom tables and can be modified but not deleted. More public views can be created and deleted.

Advanced Find

When a user is making a personal view or doing an improvised advanced search, this will be the default view of the search results. This system view is editable, but there can only be one per table. When a user is in Advanced Find and changes the columns, it can be saved and shared as a personal view. This view will then display for the user in the view selector.

Associated

Users view data on rows from the host table but can also view related tables. When selected these related data tables load on the host form in a new tab. Each table can have more than one associated view.

More system associated views exist for functions such as bulk delete operations or to display the members of a marketing list. These views have limited customization functionality.

When a user views the list of associated views for an table, they will see the system views starting with the associated views in alphabetical order, then the default public view, followed by any remaining public views in alphabetical order. Views the user has created or received through sharing will be visible alphabetically in the my views section at the bottom of the list.

Lookup

The first three columns of the lookup view are visible when a user clicks on the magnifying glass icon on the related lookup column. The system lookup view is used by default when a lookup column is placed on a form.

You may set specific lookup views and more filter criteria on a lookup column. This is known as a filtered lookup.

Quick find

This view displays search results and defines which columns are used to determine search results. There is only one quick find view per table.

Demo: View designer

DEMO

© Copyright Microsoft Corporation. All rights reserved.

- Tour of view designer
- Adding columns
- Add columns from related tables
- Sorting
- Filtering
- Quick Find columns

Demo: Compose a model-driven app

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Create an app

Add tables

Organize sitemap navigation

Module 4: Command bar

© Copyright Microsoft Corporation. All rights reserved.

Command Bar

In this module, you'll learn about how to implement actions for customized commands on the command bar in model-driven apps

In this module we will

- Create or edit modern commands
- Work with classic commands
- Use Power Fx
- Customize commands

Learning Objectives

After completing this module, you will be able to:

- 1 Create or edit modern commands
- 2 Use Power Fx
- 3 Customize commands

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Customize the command bar

<https://learn.microsoft.com/training/modules/command-bar-customize>



© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

Customize the command bar

<https://learn.microsoft.com/training/modules/command-bar-customize>

Use this module for a demo if you have the time

Common command bar scenarios

<https://learn.microsoft.com/training/modules/command-bar-scenarios>

The module is for more advanced reading

Advanced command bar concepts

<https://learn.microsoft.com/training/modules/command-bar-advanced>

Command bar

Locations

- 1 Main grid
- 2 Main form
- 3 Subgrid view
- 4 Associated view

© Copyright Microsoft Corporation. All rights reserved.

Power Apps model-driven apps implement a consistent user experience. A key component of that experience is how users are presented and can interact with available commands. As part of designing the model-driven app, a maker can customize the bars to add commands, hide commands and modify their behavior. Each command has a defined action that implements the logic that is executed when the user selects the command.

The command action can be implemented using Power Fx or JavaScript.

Commands allow users to invoke actions in the context of the data the user is working on. Commands are presented to the user in a visual collection called a command bar.

- **Main Grid** - This command bar is displayed at the top of a list of rows from a Dataverse table when the user navigates to the list from the left-hand navigation. Commands can act on any of the rows displayed in the list.
- **Main Form** - This command bar is displayed when you open a row from a table and is displayed using the main form for the table. Commands act in the context of the single row displayed on the form.
- **Subgrid View** - This command bar is visible when a user is displaying a main form for a table row, and you configure the form to have a subgrid with a related table's data. Commands act on the rows displayed in the subgrid list.
- **Associated View** - This command bar is displayed above a list of related table rows displayed from the main form. The user can display this view by selecting the related table on the main form for a row and choosing a specific table to display. Commands act in the context of one or more of the related rows displayed in the list.

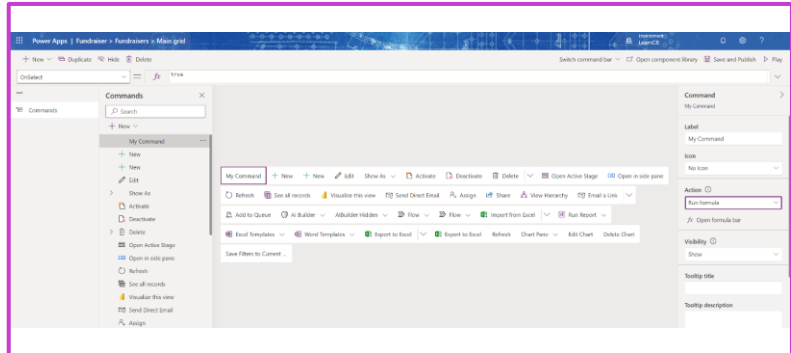
<https://learn.microsoft.com/training/modules/command-bar-customize/introduction>

<https://learn.microsoft.com/power-apps/maker/model-driven-apps/command-designer-overview>

Command bar designer

Edit in model-driven app

- Choose command bar
- Visual representation
- Add new commands
- Modify existing commands



© Copyright Microsoft Corporation. All rights reserved.

While editing a model-driven app, you can select a table and launch the visual command bar designer. After choosing which command bar you wish to modify, you'll see a visual representation of the current command bar commands. From the designer, you can add new commands and modify existing custom commands

<https://learn.microsoft.com/training/modules/command-bar-customize/create-edit>

Configuring actions

Power Fx

- **Properties**
 - OnSelect
 - Visible
- **Example use cases**
 - Access related rows
 - Create and update rows with Patch

JavaScript

- Requires JavaScript web resources
- Covered later in the course

© Copyright Microsoft Corporation. All rights reserved.

The action of a command allows you to implement custom logic that executes when the user selects the command. The command logic could perform actions like modifying values on the form or creating/modifying other data rows. A command action logic has the current selected item(s) available to it, and can use the item column values in the logic.

You can create the action logic using Power Fx expressions or by using JavaScript. For example, a command using the following Power Fx expression would navigate to a specific view for the Accounts table.

Navigate('Accounts (Views)'.My Active Accounts')

Power Fx is the low-code language that will be used across Microsoft Power Platform. You might be familiar with Power Fx if you built a Power Apps canvas app. Power Fx can also be used to implement custom logic with Power Apps model-driven command bar commands. Command bar logic can use a [subset](#) of the full Power Fx language capabilities.

There are two places you can use Power Fx when customizing a command:

- **OnSelect** - This logic executes when a command button is selected by a user.
- **Visible** - This logic is executed to evaluate if the command should be hidden or visible to the user running the app.

<https://learn.microsoft.com/training/modules/command-bar-customize/power-fx>

<https://learn.microsoft.com/training/modules/command-bar-advanced/javascript-commands>

The use of JavaScript and JavaScript libraries are covered in Extending the model-driven app user experience learning path later in the course

Demo: Command bar

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Add a command button named Cancel

Use Power Fx

Set Visibility using Show on condition from formula to only show button for active records

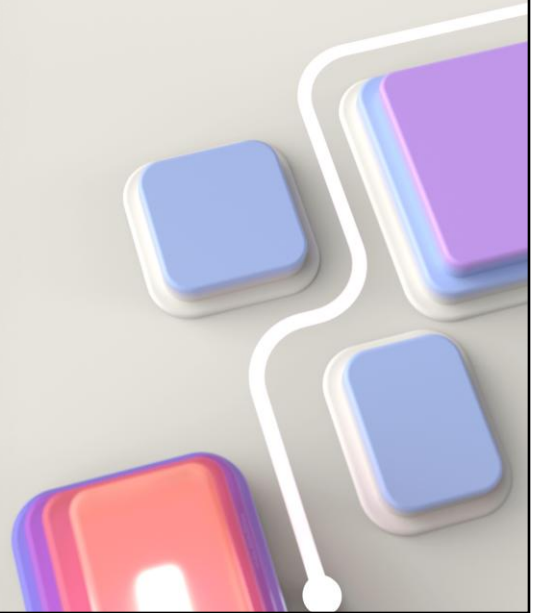
If(Self.Selected.Item.'Status Reason' = 'Status Reason (<Table>').Active, true, false)

Set OnSelect to

Patch(<Table>, Self.Selected.Item, {Status Reason: 'Status Reason (<Table>').Inactive})

Module 5: Optimizing performance

© Copyright Microsoft Corporation. All rights reserved.



Optimizing performance

In this module, you'll learn about how to optimize a model-driven app, to ensure it's performant and scalable.

In this module we will

- Discuss the importance of optimization
- Discover the different factors that can impact the performance of an application, including view and form definitions
- Review the tools at our disposal to analyze and detect performance issues

Performance optimization

When analyzing performance of Dataverse, it is important to understand areas affecting the performance of an app

- 1 App definition
- 2 Form / View structures
- 3 Server-side logic
- 4 Client-side logic
- 5 Security privileges

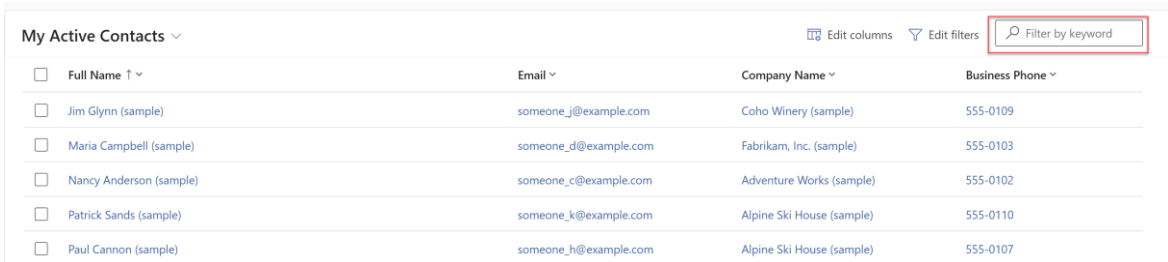
© Copyright Microsoft Corporation. All rights reserved.

When analyzing performance of Dataverse, it is important to understand areas affecting the performance of an app:

- **App definition:** A single, monolithic app that covers all required functionality areas within a business may be easier to manage but could lead to potential performance problems. Consider carefully the number of tables, views, forms and commands added to the app and explore the possibility to split the app functionality into multiple, smaller applications.
- **Form / view structures:** A form or view that contains all attributes will never be wholly performant. What may initially be helpful to users could cause frustrations in the future. The definitions of these objects should always be tailored to contain the minimum required information for the app to function. This is particularly important with views and, with proper user training on Advanced Find functionality, users can be encouraged to create additional ad hoc views for specific scenarios they face.
- **Server-side logic:** Arguably the powerfulllest feature of Dataverse is the ability to enforce our business logic as the underlying database transaction executes. Whilst this does unlock numerous possibilities, badly implemented logic could lead to user frustrations, as our custom code or workflows block users from proceeding from one screen to the next. These kinds of complex logic should always be considered carefully, balancing business requirements and performance considerations in equal measure.
- **Client-side logic:** JavaScript offers a powerful way to modify or perform advanced logic at the form level but, if implemented poorly, could be a source of performance issues or user frustration. Developers should always avoid implementing “blocking” logic, and favour asynchronous execution patterns
- **Security privileges:** If modelled out correctly, security privileges can act as an effective mechanism to “declutter” an application and ensure users are only presented with the information they require. This could avoid the need of creating separate model-driven apps for specific scenarios.

App performance

Design of the app itself can greatly affect performance



The screenshot shows a table titled "My Active Contacts" with a search bar and filters. The search bar is highlighted with a red box and contains the text "Filter by keyword". The table has four columns: Full Name, Email, Company Name, and Business Phone. The data is as follows:

☐ Full Name ↑	Email	Company Name	Business Phone
☐ Jim Glynn (sample)	someone_j@example.com	Coho Winery (sample)	555-0109
☐ Maria Campbell (sample)	someone_d@example.com	Fabrikam, Inc. (sample)	555-0103
☐ Nancy Anderson (sample)	someone_c@example.com	Adventure Works (sample)	555-0102
☐ Patrick Sands (sample)	someone_k@example.com	Alpine Ski House (sample)	555-0110
☐ Paul Cannon (sample)	someone_h@example.com	Alpine Ski House (sample)	555-0107

© Copyright Microsoft Corporation. All rights reserved.

Design of the app itself can greatly affect performance. One of the strengths of the Power Platform is ability to quickly build robust small apps targeting specific users and specific functionality. Instead of a monolithic app that is more likely to suffer from performance issues you now have the ability to design several separate apps. This separation ensures that each app can be analyzed and tuned independently without too much impact on the others.

Design techniques for individual components may also affect performance of an app.

To improve form performance, design them so that only the information essential for the users is included. If information required depends on the user's role, consider creating multiple forms instead of one monolithic form that includes every single column. Hiding tabs and sections that are not required to be immediately visible can also improve form responsiveness and performance.

One of the most common sources of performance issues on the forms is custom scripts added by developers. Good collaboration within the team is a key to identify and resolve any performance bottlenecks introduced during the development.

To improve performance of the table views, limit the number of columns displayed only to those required by the business. As often the case, it's a balance between the responsiveness and bringing enough information to improve usability of the view.

Customize your views so that they bring back an actionable number of rows. For example, Active Contacts view may contain hundreds of thousands of rows while My Active Contacts will limit that view only to the contacts that are owned by the user. At the same time consider limiting the number of rows per page so that the views return much more quickly.

Optimizing the quick search view only to include columns that makes sense to search will make a noticeable performance impact on searches within the views.

App performance

When a user searches for rows within a view, Dataverse performs a query that searches only predefined columns

Quick Find Active Accounts >

View

Name *

Quick Find Active Accounts

Description

Sort by ...

↑ Account Name X

Then sort by ... v

Filter by ...

Status is 0 X

▽ Edit filters ...

Find by ... ⓘ

Account Name X

Account Number X

Email X

Main Phone X

Other Phone X

🔍 Edit find table columns ...

© Copyright Microsoft Corporation. All rights reserved.

When a user searches for rows within a view, Dataverse performs a query that searches only predefined columns. Which columns are searched is defined by the Find Columns settings in the system Quick View.

The fewer number of columns that are selected, the faster quick search will be. Balance the number of find columns with the effectiveness of the search.

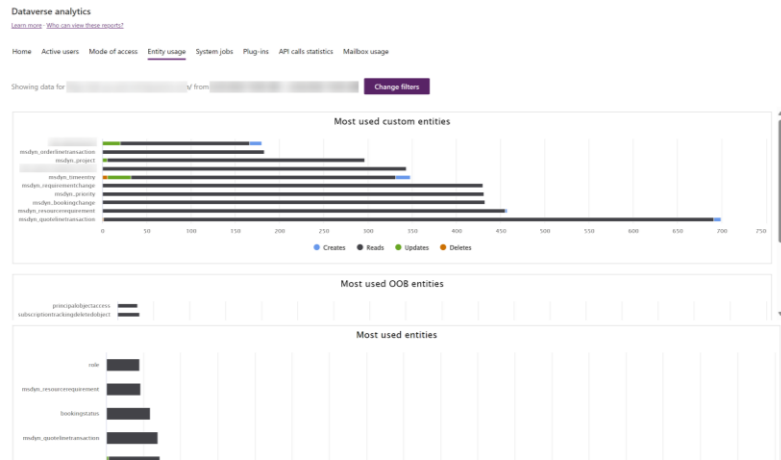
Real-time workflows can be very effective but, if designed poorly, will affect the performance of the most common operations such as creating or updating a row. Consider converting poorly performing or long running workflows to background ones where the impact on the system will be smaller.

Background workflows create a log row when an instance of the workflow is executed. The default setting is to delete log entries for successfully executed workflows. If this setting is changed to keep the entries, jobs log will grow and may reach the point where it affects performance of the asynchronous services. Only use this flag for troubleshooting and avoid leaving it on in production.

Custom extensions such as plugins are the common source of the performance issues. If a plug-in is taking on average more than 30 seconds to fully execute, you should review the logic and adapt accordingly. Consider creating an Azure aware plug-in in these scenarios, to offload the processing from Dataverse and to leverage additional compute power.

Analytics

- Dataverse analytics are available via the Power platform admin center:
- <https://aka.ms/ppac>



© Copyright Microsoft Corporation. All rights reserved.

Dataverse analytics are available via the Power platform admin center <https://aka.ms/ppac>. Various statistics can help identifying potential areas for deeper performance analysis.

The Entity Usage tab contains information about most used out-of-the-box and custom tables. Identifying the most used tables helps concentrate the optimization efforts on the parts of the app that matter.

Analytics

The Dataverse tab in the Capacity area provides several storage statistics across all your tenant instances as well as a breakdown of the storage within a specific instance

Capacity

Summary

Dataverse

Microsoft Teams

Add-ons

Trial

Finance and operations

See where storage is being used across your environments. If you're added to an environment as an admin, it will be included in this list. Click on an environment to see the trends. [Learn more](#)

Download

Search

Environment	Details	Type	Pay-as-you-go	Database usage	File usage	Log usage	Last updated
		Production	No	17.72 GB	37.67 GB	6.25 GB	
		Sandbox	No	10.67 GB	36.61 GB	134.22 MB	
		Sandbox	No	9.71 GB	35.19 GB	906.31 MB	
		Sandbox	No	5.03 GB	10.81 GB	681.59 MB	
		Sandbox	No	4.86 GB	8.83 GB	224.14 MB	
		Support	No	0 MB	0 MB	0 MB	

© Copyright Microsoft Corporation. All rights reserved.

The Dataverse tab in the Capacity area provides several storage statistics across all your tenant instances as well as a breakdown of the storage within a specific instance.

This can help identify tables with excessive storage use that would be a primary target for further performance investigations.

Other statistics are also available that may be useful. Plug-ins statistics can assist in identifying troublesome plugins, for example those with excessive average execution time.

Power Apps Analytics help with performance analysis by providing detailed statistics on Service Performance across various connectors used in your app.

Additional information such as overall app usage, location statistics, or errors seen by end users can be extracted from other tabs. While not directly performance related, this information can be useful in pinpointing overall problematic areas that may be caused by poor app performance such as specific region, specific app, etc.

Learning Path 2 practice labs



Power Apps

Lab 2: Model driven app

© Copyright Microsoft Corporation. All rights reserved.

In this lab we will continue to build on top of the Microsoft Dataverse components created in the previous lab. We will now build a Power Apps model-driven app to allow the office staff manage records for the inspectors and the inspectors to manage their own records as needed.

Summary



- Model-driven app design is an approach that focuses on quickly adding components to apps. These components include dashboards, forms, views, and charts
- Little or no code is required to build model-driven apps
- Model-driven design uses metadata-driven architecture so that designers can customize their apps
- Buttons can be added to the command bar to invoke actions with Power Fx or JavaScript

© Copyright Microsoft Corporation. All rights reserved.

Defining and enforcing consistent business processes is a key aspect of model-driven app design. Consistent processes help ensure that your app users can focus on their work and not have to remember to perform a set of manual steps.

Let's review what you've learned:

- Model-driven app design is an approach that focuses on quickly adding components to apps. These components include dashboards, forms, views, and charts.
- Little or no code is required to build model-driven apps.
- Model-driven design uses metadata-driven architecture so that designers can customize their apps.

Modules

1. Introduction to model-driven apps
2. Design and create model-driven apps
3. Configure forms
4. Configure views
5. Configure charts
6. Configure dashboards



© Copyright Microsoft Corporation. All rights reserved.